

**SIMATS ENGINEERING
ASSESSMENT TOOL 1**

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks:50

Mode: In Class

Hours : 60 mts

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

I H. VAISHNAV (19252125) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/ 20	/ 25	/ 20	/ 15	/ 100

1. Trace the processing of the input Expression ..

Expression:

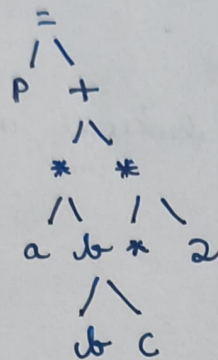
$$p = (a * b) + (b * c) * 2$$

(a) Tokens generated during lexical analysis

<u>Lexeme</u>	<u>Token</u>
p	Identifier
=	Assignment operator
(	Left parenthesis
a	Identifier
*	Multiplication operator
b	Identifier
)	Right parenthesis
+	Addition operator
(	Left parenthesis
b	Identifier
*	Multiplication operator
c	Identifier
)	Right Parenthesis
*	Multiplication operator
2	Integer constant

Total Tokens = 15

(b) Syntax Tree:



(c) Intermediate Code:

$$t_1 = a * b$$

$$t_2 = b * c$$

$$t_3 = t_2 * 2$$

$$t_4 = t_1 + t_3$$

$$P = t_4$$

(d) Optimized Intermediate Code:

No common subexpression exists,
so only simple optimization is applied.

$$t_1 = a * b$$

$$t_2 = b * c$$

$$t_2 = t_2 * 2$$

$$P = t_1 + t_2$$

It reduces one temporary variables.

(e) Significance of Final Target code:

The target code is the machine or assembly code generated by the compiler. It is optimized for an efficient execution on the processor. It minimizes memory usage, improves execution speed and can be directly executed by the computer.

2. (a) $(aa^*)^*$

Language: All strings consisting of one or more 'a's.

Five valid strings :-

1. a
2. aa
3. aaa
4. aaaa
5. aaaaa

(b) $(b^*)^*abb$

Language: any number of b's followed by abb.

Five valid strings :-

1. abb
2. babb
3. bbabb
4. bbbabb
5. bbbbabb

(c) $(ab)^*aba(a+b^*)^*$

Language: strings containing the substring aba.

Five valid strings :-

1. aba
2. abaa
3. abab
4. aababa
5. bbabaa

(d) $(ab)^*$

Language :- Zero or more repetitions of "ab".

Five valid strings :-

1. ϵ (Empty string)
2. ab
3. abab
4. ababab
5. abababab

(e) $(aba)^* + (ab)^*$

Language :- union of strings formed by repeating "aba" and strings containing any number of a's followed by any number of b's.

Five valid strings :-

1. aba
2. abaaba
3. aabbb
4. aaab
5. bbb

3. (i) Number of Tokens :-

program :-

```
main ()
{
    int a = 10;
    char b = "abc";
    int c = 80;
```

2. $a = \&x;$

* Assigning address of integer to the character pointer.

* Lexically valid but semantically incorrect.

Lexical error identified :-

$\&x$ ab is an invalid token.

4. (i) Strings having an even number of 0's.

regular expression :-

$$(1^* 01^* 01^*)^*$$

(ii) Strings containing at least one 1 :-

regular expression :-

$$0^* 1 0^*$$

or more generally

$$0^+ 1 0^+$$

(iii) Both number of 0's and 1's are odd :-

Regular expression :-

$$1^* (01^* 01^*)^* 01^*$$

(Any equivalent expression accepted.)

(iv) Strings starting with 1 and ending with 0.

Regular expression :-

$$1 0^+ 0$$

(v) Strings without consecutive 1's

Regular expression: $(0+10)^* 1?$

```
char d = "xyz";
```

```
in /* comment */ m = 40.5;
```

```
}
```

Token count

Statement

Tokens

```
main ()
```

4

```
{
```

1

```
int a = 10;
```

5

```
char b = "abc";
```

5

```
int c = 30;
```

5

```
char d = "xyz";
```

5

```
in /* comment */ m = 40.5;
```

6

```
}
```

1

Total Tokens = 32

(ii) Lexical Errors:-

```
void main ()
```

```
{
```

```
int x = 10, y = 20;
```

```
char * a;
```

```
a = &x;
```

```
x = 1xab;
```

```
}
```

Errors:-

1. $x = 1xab;$

* Invalid hexadecimal constant

* correct form $\Rightarrow x = 0xAB;$

5.

Input string ::

int var123 = 12345;

float var456 = 123.45;

if (var123 == var456)

{
return;

}

(a) Number of Tokens :-

<u>Token</u>	<u>Type</u>
int	keyword
var123	identifier
=	operator
12345	integer literal
;	separator
float	keyword
var456	identifier
=	operator
123.45	floating literal
;	separator
if	keyword
(	separator
var123	identifier
==	operator
var456	identifier
)	separator

{ separator
return keyword
; separator
} separator

Total Tokens = 20.

(b) categorised Tokens :-

Keywords :-

* int
* float
* if
* return

operators :-

* =
* =
* ==

separators :-

* ;
* ;
* (
*)
* {
* ;
* }

Identifiers :-

* var 123
* var 456
* var 123
* var 456

Integer literals :-

* 12345

Floating - point literals

* 123.45

(c) FSM State Transitions

Estimated Transitions :-

<u>Token Type</u>	<u>Approx. Trans.</u>
-------------------	-----------------------

Keywords (4)	18
--------------	----

Identifiers (4)	24
-----------------	----

Integer Literal (1)	5
---------------------	---

Floating Literal (1)	6
----------------------	---

operators (3)	4
---------------	---

separators (7)	7
----------------	---

Total Estimated FSM

Transitions.

$$18 + 24 + 5 + 6 + 4 + 7 = 64$$

Therefore, 64 state transitions are required to tokenize the input string.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/for mula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited unclear or interpretation	No interpretation